Hello, Python

Course: Kaggle Learn · Instructor: Colin Morris · Lesson 1 of 7

1. Introduction to Python

Python is a high-level programming language known for its readability and simple syntax. It is widely used in data science, machine learning, web development, automation, and more.

This course covers essential Python skills needed to begin working with data science.

- Prerequisites: Some previous coding experience is helpful.
- First-time coders: Kaggle's Intro to Programming course is recommended.

2. Variables and Variable Assignment

A variable is a name that refers to a value. The assignment operator `=` assigns a value to a variable.

```
spam_amount = 0

print(spam_amount)    # Output: 0
```

Here:

- `spam_amount` is the variable name.
- `=` is the assignment operator.
- `0` is the assigned value.

Reassignment

An existing variable can be assigned a new value.

```
spam_amount = 0

spam_amount = spam_amount + 4

print(spam_amount)
# Output: 4
```

How it works:

1. Python evaluates the right-hand side: `0 + 4`.
2. The result, `4`, is assigned to `spam_amount`.
3. The variable now refers to the new value.

Important characteristics of Python variables

- Variables do not need to be explicitly declared before assignment.
- You do not need to specify a variable's data type when creating it.
- A variable can be reassigned to a value of a different type.

```
x = 10
x = "Hello"
x = True
```

The variable `x` refers to an integer, then a string, and then a Boolean value.

3. Functions and Function Calls

A function is a reusable block of code that performs a task.

Functions are called by writing their name followed by parentheses `()`.

```
print("Hello, Python!")
```

Output:

```
Hello, Python!
```

Here, `print()` is a built-in function that displays information on the screen.

Arguments

The values passed into a function are called arguments.

```
print(10)
print("Hello")
print(3 + 4)
```

Output:

```
10
Hello
```

```
7
```

The `type()` function

`type()` returns the type of an object.

```
print(type(10)
)# <class 'int'>

print(type(19.95))
# <class 'float'>

print(type("Python"))
# <class 'str'>
```

This function is useful when you need to determine what kind of value a variable contains.

4. Comments

Comments are notes written in code for humans to read. Python ignores comments when executing a program.

Single-line comments begin with the `#` symbol.

```
# This is a comments
pam_amount = 4  # Store the number of spam servings
print(spam_amount)
```

Uses of comments:

- Explain what a section of code does.
- Make code easier to understand.
- Leave reminders or clarifications for future reference.

5. Conditional Statements and Indentation

An `if` statement executes a block of code only when a specified condition is true.

```
spam_amount = 4
if spam_amount > 0:
    print("But I don't want ANY spam!")
print("This line always executes.")
```

Output:

But I don't want ANY spam!

```
This line always executes.
```

Understanding the syntax

- `if` introduces a conditional statement.
- `spam_amount > 0` is the condition.
- `:` marks the beginning of the code block.
- Indentation identifies which statements belong to that block.

Indentation in Python

Python uses indentation, typically four spaces, to define code blocks instead of curly braces `{}`.

```
if spam_amount > 0:
    print("Inside the if block")
print("Outside the if block")
```

The first `print()` is indented and belongs to the `if` block. The second is not indented and executes independently of that condition.

Remember: Incorrect indentation can cause an `IndentationError` or change the program's behavior.

6. Strings

A string (`str`) is a sequence of characters used to represent text.

Strings can be enclosed in single or double quotation marks.

```
name = "Python"
message = 'Hello, world!'
print(name)
print(message)
```

Both quotation styles work. Choose the one that makes your string easiest to write.

For example, double quotes are convenient when the text contains an apostrophe:

```
message = "But I don't want ANY spam!"
```

Alternatively, you can use single quotes if you escape the apostrophe:

```
message = 'But I don\'t want ANY spam!'
```

String multiplication

Python allows a string to be multiplied by an integer. This repeats the string that many times.

```
viking_song = "Spam " * 4
print(viking_song)
```

Output:

```
Spam Spam Spam Spam
```

The trailing space exists because `"Spam "` contains a space after the word.

```
print("Ha" * 3)       # HaHaHa
```

Important: String multiplication is different from numerical multiplication. It repeats text rather than calculating a numeric product.

7. Numbers and Data Types

Python provides several numeric data types. Two important ones are `int` and `float`.

Integer (`int`)

An integer is a whole number without a fractional component.

```
age = 19
spam_amount = 0
print(type(age))             # <class 'int'>
```

Examples: `0`, `5`, `-12`, `100`.

Floating-point number (`float`)

A float represents a number that can have a fractional component.

```
weight = 72.5
price = 19.95
print(type(price))             # <class 'float'>
```

Examples: `3.14`, `-0.5`, `19.95`.

Difference between `int` and `float`

Feature	`int`	`float`
Represents	Integers	Floating-point numbers
Example	`5`	`5.0`
Decimal component	No fractional component	Can represent fractional values
Type	`int`	`float`

Although `5` and `5.0` have the same numerical value, they have different types.

8. Arithmetic Operators

Python supports the standard mathematical operations using arithmetic operators.

Operator	Name	Description	Example	Result
`+`	Addition	Adds two values	`5 + 2`	`7`
`-`	Subtraction	Subtracts the second value from the first	`5 - 2`	`3`
`*`	Multiplication	Multiplies two values	`5 * 2`	`10`

`/`	True division	Divides two values	`5 / 2`	`2.5`
`//`	Floor division	Floors the quotient	`5 // 2`	`2`
`%`	Modulus	Returns the remainder	`5 % 2`	`1`
`**`	Exponentiation	Raises a number to a power	`5 ** 2`	`25`
`-x`	Negation	Reverses the sign	`-5`	`-5`

Examples

```
a = 10
b = 3

print(a + b)   # 13
print(a - b)   # 7
print(a * b)   # 30
print(a / b)   # 3.333...
print(a // b)  # 3
print(a % b)   # 1
print(a ** b)  # 1000
```

9. True Division vs. Floor Division

Python provides two division operators: `/` and `//`.

True division (`/`)

True division returns a floating-point result, even when the quotient is a whole number.

```
print(5 / 2)# 2.5
print(6 / 2)# 3.0
```

Floor division (`//`)

Floor division rounds the quotient down to the greatest integer less than or equal to the result.

```
print(5 // 2)# 2
print(6 // 2)# 3
```

For positive numbers, this often looks like removing the fractional part. However, for negative numbers, floor division rounds toward negative infinity.

```
print(-5 // 2)# -3
```

Modulus (%)

The modulus operator returns the remainder associated with division.

```
print(5 % 2)# 1
print(10 % 3)# 1
print(8 % 2)# 0
```

Common use: Checking whether a number is even or odd.

```
number = 7
print(number % 2 == 0)# False
```

An even number has a remainder of `0` when divided by `2`.

10. Order of Operations

Python follows standard mathematical precedence rules, commonly remembered through PEMDAS:

1. P — Parentheses
2. E — Exponentiation
3. M/D — Multiplication and division, from left to right
4. A/S — Addition and subtraction, from left to right

```
print(8 - 3 + 2)# 7
print(-3 + 4 * 2)# 5
print(2 + 3 * 4)# 14
```

In the last example, multiplication happens before addition:

$$2 + (3 \times 4) = 14$$

Using parentheses

Parentheses change the order in which operations are evaluated.

```
print((2 + 3) * 4)# 20
print(2 + 3 * 4)# 14
```

Use parentheses when you want to make the intended calculation explicit.

Example: Calculating height

Suppose a person's height is `190` cm and their hat is `25` cm tall.

Incorrect calculation:

```
hat_height_cm = 25
my_height_cm = 190
total_height_meters = hat_height_cm + my_height_cm / 100
print(total_height_meters)# 26.9
```

Python divides `my_height_cm` by `100` before adding the hat height.

Correct calculation:

```
total_height_meters = (hat_height_cm + my_height_cm) / 100
print(total_height_meters)# 2.15
```

The parentheses ensure that the heights are added before converting centimeters to meters.

11. Built-in Functions for Working with Numbers

Python provides built-in functions that simplify common numerical operations.

`min()` and `max()`

- `min()` returns the smallest of the supplied arguments.
- `max()` returns the largest of the supplied arguments.

```
print(min(1, 2, 3))# 1
print(max(1, 2, 3))# 3
```

They can also accept a list or another iterable.

```
numbers = [8, 3, 12, 5]
print(min(numbers))  # 3
print(max(numbers))  # 12
```

abs()

`abs()` returns the absolute value of a number, representing its distance from zero.

```
print(abs(32))# 32
print(abs(-32))# 32
print(abs(-4.5))# 4.5
```

float()

`float()` converts a compatible value to a floating-point number.

```
print(float(10))# 10.0
print(float("3.14"))# 3.14
```

int()

`int()` converts a compatible value to an integer.

```
print(int(3.33))# 3
print(int(-3.33))# -3
print(int("807") + 1)# 808
```

Important: When converting a float to an integer, `int()` truncates the fractional part toward zero. It does not round to the nearest integer.

For example:

```
print(int(3.99))   # 3
print(int(-3.99))  # -3
```

The string `"807"` is converted to the integer `807`, which can then be used in arithmetic.

Quick reference

Function	Purpose

`print(x)`	Displays a value
`type(x)`	Returns the type of a value
`min(...)`	Finds the minimum value
`max(...)`	Finds the maximum value
`abs(x)`	Returns the absolute value
`float(x)`	Converts a compatible value to `float`
`int(x)`	Converts a compatible value to `int`, truncating floats toward zero

12. Common Mistakes to Remember

a. Confusing `=` with equality

   `x = 5` assigns a value. `x == 5` compares a value and returns a Boolean.

b. Ignoring indentation

   Code indented under `if` belongs to its code block. A line outside the block executes independently of that condition.

c. Confusing `/` and `//`

   `5 / 2` gives `2.5`, whereas `5 // 2` gives `2`.

d. Assuming `int()` rounds numbers

   `int(3.9)` returns `3`, not `4`. It truncates toward zero.

e. Forgetting operator precedence

   `2 + 3 * 4` evaluates to `14`, not `20`. Use `(2 + 3) * 4` to get `20`.